

Challenge0

InventoryMaster - Hackathon Challenges

Overview

This template provides the foundation for an E-Commerce Inventory Management System. Your task is to implement the missing functionality to make all test cases in `challenge0.test.js` pass.

Challenge Requirements

Challenge 0: Product Management System

You need to implement the following core functionalities:

a Homepage Title Configuration

Files to modify:

- `client/index.html`

Challenges:

- Change the page title from “TODO: Set Inventory System Title” to “InventoryMaster”
- Simple HTML modification to get started

b Navigation Links Configuration

Files to modify:

- `client/index.html`

Challenges:

- Add `id="signUpLink"` to the signup link and set `href="signup.html"`
- Add `id="loginLink"` to the login link and set `href="login.html"`
- Add `class="btnGetstarted"` to the “Get Started” button and set `href="login.html"`

c SKU Uniqueness Validation

Files to modify:

- `src/repositories/productRepository.js` - `getProductBySku()` function
- `src/controller/productController.js` - `createProduct()` function

Challenges:

- Implement SKU lookup functionality
- Add SKU uniqueness check before product creation
- Implement case-insensitive SKU validation (e.g., "TEST-001" and "test-001" should be considered the same)
- Return appropriate error message for duplicate SKUs

d ❌ Stock Update Operations**Files to modify:**

- `src/repositories/productRepository.js` - `updateProduct()` function
- `src/controller/productController.js` - `updateStock()` function

Challenges:

- Implement dynamic product update with flexible field updates
- Add input validation for stock operations (must be integers, no decimals allowed)
- Calculate new stock quantities based on add/subtract operations
- Prevent negative stock scenarios
- Validate operation type ('add' or 'subtract' only)
- Prevent zero-quantity operations

e ❌ Reorder Level Detection (ADVANCED)**Files to modify:**

- `src/repositories/productRepository.js` - `getProductsNeedingReorder()` function
- `src/controller/productController.js` - `getProductsNeedingReorder()` function

Challenges:

- Query products where `stock_quantity <= reorder_level`
- Filter only active products
- **TRICKY PART:** Handle user isolation properly (only return products belonging to the authenticated user)

- Sort results by priority (lowest stock-to-reorder ratio first)
- Correctly handle multiple users accessing their own products

f Comprehensive Product Management

Files to modify:

- `src/repositories/productRepository.js` - `createProduct()` , `getProductById()` , `updateProduct()` functions
- `src/controller/productController.js` - `createProduct()` , `updateProduct()` functions

Challenges:

- Complete product CRUD operations
- Handle SQL parameter binding correctly (especially for optional fields like `supplierId`)
- Implement proper error handling and validation
- Enforce resource ownership (users can only access their own products)
- Support partial updates (only update fields that are provided)
- Implement proper category validation

g Password Complexity Validation

Files to modify:

- `src/models/user.js` - `validateSignup()` function
- `src/controller/authController.js` - `signup()` function

Challenges:

- Implement password complexity validation
- Require minimum 8 characters
- Require at least one uppercase letter
- Require at least one lowercase letter
- Require at least one number
- Require at least one special character
- Return appropriate error messages for different validation failures

h Smart Security Question System (ADVANCED DIFFICULTY)

Files to create/modify:

- `src/routes/authRoutes.js` - Add new route for security questions

- `src/controller/authController.js` - Add security question handling
- `src/models/user.js` - Update model for security questions

Challenges:

- Implement a security question system for password recovery
- Allow users to set security questions during signup
- Create an endpoint to verify security question answers
- **TRICKY PART:** Implement a special answer verification logic based on last name:
 - Last names starting with T-Z: Answers must be in UPPERCASE
 - Last names starting with N-S: Answers must be REVERSED
 - Last names starting with G-M: First and last letters must be SWAPPED
 - Last names starting with A-F: Answers used as-is
- The system must verify answers according to these rules
- The rule is not explicitly stated in the API but must be discovered

Good luck! 🍀